QA CASE STUDY · BANKING

From FSD to Automation

How I tested a bond e-statement platform for a multinational bank: requirement analysis, 139 test cases and a keyword-driven Katalon framework.

Mulla Shadra

Software Quality Assurance

Katalon Studio · Groovy · SQL · Manual + Automation

01 · THE PROJECT

Project at a glance

A monthly statement engine for bond trades. It matches three raw files, builds **encrypted PDF statements**, holds them for a verifier's review (PAT), then emails them to customers.

My role: QA. I analyzed the FSD, extended test scenarios with negative and edge cases, executed functional and integration testing, and built the automation framework.

Client: multinational bank

Domain: capital markets

2025 – 2026

6

functional modules in the FSD

139

test cases, 1 in 3 negative

14

automated test cases in Katalon

38

reusable custom keywords

02 · SYSTEM UNDER TEST

From raw file to customer inbox

Raw files in
RPT, RPKBP 15 and SCAACNT, from a folder or upload

→

Process and match
Files joined on BPID and SCA ID

→

PDF statements
Messages and inserts added, encrypted per customer

→

PAT review
A PIC approves a random sample first

→

Send and track
Retry on soft bounce, delivery and TAT reports

Modules covered by the FSD

Dashboard | Customer Service | Statement Preparation | E-Statement Setup | Reports | Background services

03 · RISK AREAS

Where the risk lived

Access control

Every page is permission-based. Users only see what their role allows.

Maker-checker

Custom messages and email templates need a second user to approve. Ad hoc requests do not.

File validation

Raw files must be .RPT, .CSV, .XLS or .XLSX. Criteria uploads accept .xls or .xlsx only.

Three-file match

RPT, RPKBP 15 and SCAACNT must line up on BPID and SCA ID before any PDF is built.

PAT gate

A PIC reviews a random sample. Sending to customers unlocks only after the set share is approved.

Suppression and security

Suppression lists go in before PAT. Every PDF is encrypted with a per-customer password.

04 · APPROACH

Four steps from requirement to regression

01

Analyze

Turned FSD conditions and screen elements into testable rules, tracked across 4 versions (v1.0 – v1.3).

02

Design

Wrote positive, negative, boundary and role-based scenarios for each module.

03

Execute

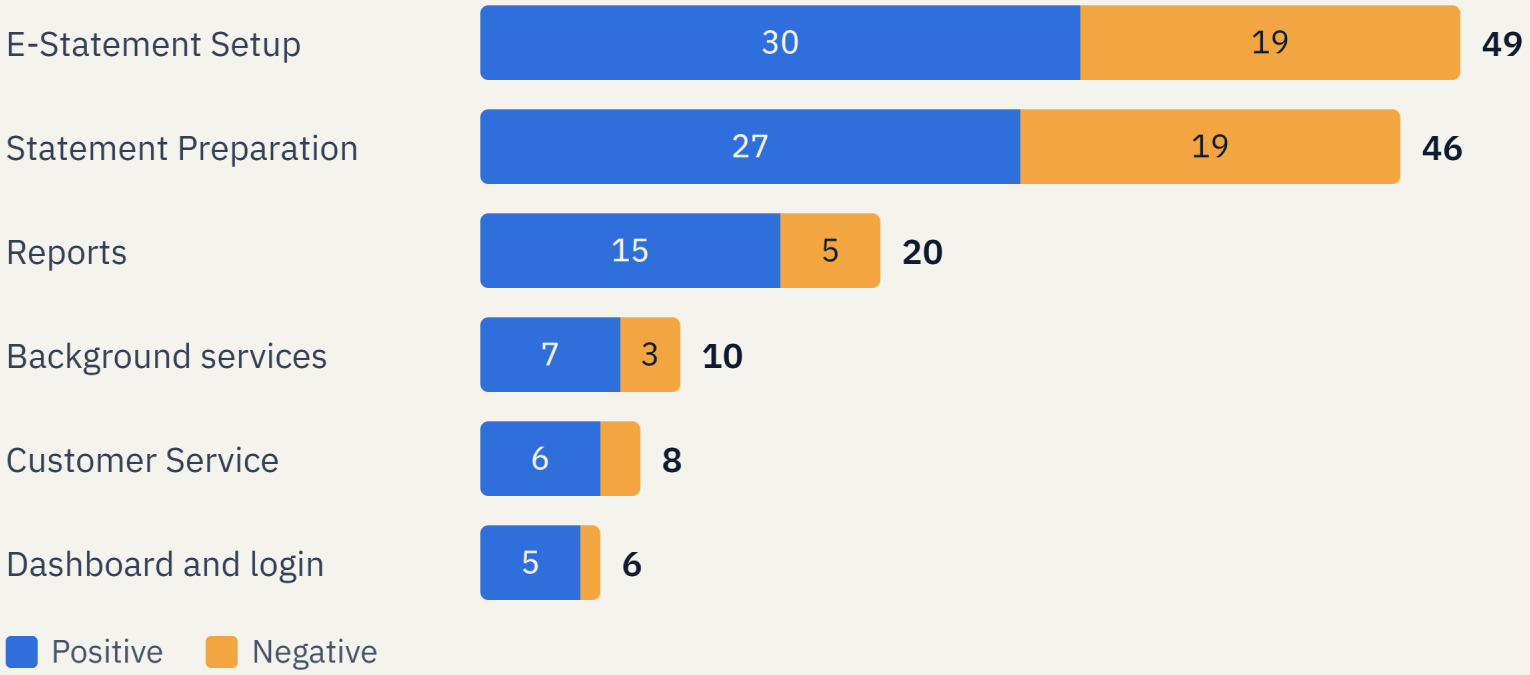
Validated from UI to database and logged defects with reproduction steps and impact.

04

Automate

Automated stable, frequently repeated flows as keyword-driven regression in Katalon.

139 test cases across 6 modules



90
positive scenarios

49
negative scenarios

About **1 in 3** cases checks what the system must reject.

Business rules turned into checks

Module	Scenario	Expected result	Type
Custom Message	Create a message with a code that already exists	Popup "Data Already Existing" blocks the save	Negative
Suppression	Upload a suppression list in the wrong format	Popup "File type must .xls or .xlsx"	Negative
Email Template	Save CIF criteria without a value	Popup "Criteria value cannot be empty"	Negative
Raw Processing	Process raw files that contain invalid data	Run completes and the file is marked Failed with a remark	Negative
Ad hoc Statement	Reprint to staff with password on	Statement emailed to the PIC as a password-protected PDF	Positive

Following one run through the database

1. **Errors and service logs**
   EXCEPTION_LOG, SYSTEM_LOG
2. **Raw file picked up**
   FILE_PROCESSING
3. **Rows loaded to staging**
   3 staging tables, one per file
4. **Statement PDFs generated**
   STATEMENT_DOC
5. **PAT review before release**
   STATEMENT_PAT_EMAIL
6. **Sent to customers, delivery logged**
   EMAIL_SEND, EMAIL_HISTORY

```
BEGIN TRAN
DECLARE @runStart DATETIME = '<run start>'

-- errors raised during the run
SELECT TOP 5 * FROM EXCEPTION_LOG WITH (NOLOCK)
WHERE LOG_DATE > @runStart

-- trace each email to its delivery status
SELECT EH.* FROM EMAIL_HISTORY EH WITH (NOLOCK)
JOIN EMAIL_SEND ES WITH (NOLOCK)
ON EH.EMAIL_SEND_ID = ES.EMAIL_SEND_ID
WHERE ES.CREATE_DATE > @runStart
ROLLBACK TRAN
```

One timestamp scopes every layer to the current test run.

Joins link each upload to its emails and delivery result.

Read-only: NOLOCK reads inside a rolled-back transaction.

An issue log I designed for the project

Open → **Bug Fixing** → **Fixed** → **Retest** → **Closed**

Failed retest: Failed, then Re-Open, then back to Bug Fixing

On hold: TBC, Blocking, Cancelled

Status	Total Issue	Testing Type	
		Positive Test	Negative Test
Open	0	0	0
Bug Fixing	0	0	0
Fixed	0	0	0
Testing	0	0	0
Passed	0	0	0
Failed	0	0	0
Re-Open	0	0	0
TBC	0	0	0
Blocking	0	0	0
Cancelled	0	0	0
Closed	0	0	0
Total	**0**	**0**	**0**

No	Issue ID	Log Date	Severity	Module	Sub Modu	Issues	Testing Type	Status	Developer	QA	Notes	Step to reproduce	Expected	Actual	Evidence

One sheet per environment (VIT, UAT, Production), each with a status summary on top and one row per issue: steps to reproduce, expected vs actual, and evidence.

09 · HOW THE FRAMEWORK GREW

From recorded clicks to a Page Object Model

STEP 1 · KATALON

Record and playback

Quick to start, but every recorded test repeated the login, the menus and the hardcoded data.

STEP 2 · SELENIUM

Learning the Page Object Model

Exploring Selenium WebDriver, with AI as a study partner, taught me POM: one class per page, locators kept apart from test logic.

STEP 3 · BACK TO KATALON

POM with custom keywords

I applied the idea in Katalon: locators in the Object Repository, keyword classes as page objects, tests as business steps.

The takeaway: good test design carries across tools. The concept came from one tool, and I rebuilt it in the tool the project used.

A Page Object Model, built with Katalon keywords

Test Cases	business steps only, no locators
Page actions	one keyword class per page
Page verifications	checks kept apart from actions
Shared components	login, menus, criteria, save, approve
Object Repository	locators per page, parameterized

4
modules automated

14
test cases

16
keyword classes

38
custom keywords

One keyword, built from small reusable steps

```groovy
// EmailTempAction.groovy
@Keyword
def addNewEmailTemplate(code, name, emailType,
emailSubject, periodFrom, periodTo) {
WebUI.click(findTestObject('common/btn_Add'))
openForm()
fillBasicInfo(code, name, emailType, emailSubject)
fillEmailBody()
handleActivePeriod(periodFrom, periodTo)
handleCriteria()
saveForm()
}
```

Page steps stay private

White lines are private helpers of this page: open the form, fill it in.

Shared steps are reused

Blue lines call the common package, so other pages use the same period, criteria and save logic.

Locators take parameters

```
//td[normalize-space()='${code}']
```

One test object finds any table row by its template code.

Test scripts that read like the spec

```
// Add Email Template: lands as "New"
WebStarter.openWeb()
Login.login()
Navigation.goToEmailTemplate()
EmailTempAction.addNewEmailTemplate(
code, name, "Regular Statement",
subject, periodFrom, periodTo)
EmailTempVerification
.verifyApprovalStatus(code, "New")
```

One line per business step

Each script is a handful of keyword calls that a non-coder can follow.

Actions apart from checks

Every module has an Action class and a Verification class.

Shared helpers

Active period, criteria and save handlers are reused across modules.

13 · END-TO-END SUITE

One suite runs the whole approval lifecycle



Nested iframes

Switches from the main frame to the pop-up form to the rich-text editor, then back.

Rich-text body

Email body filled through a JavaScript executor, then personalization tags inserted.

Alert chains

Confirm and success alerts handled once, in shared save, approve and reject handlers.

Dynamic locators

Test objects take the template code as a parameter to find the right table row.

Same test, two ways to automate it

RECORD AND PLAYBACK · repeated in every test

```
WebUI.openBrowser('')
WebUI.navigateToUrl('https://app-url/login')
WebUI.setText(findTestObject('Page_Login/input_User'), 'maker01')
WebUI.setEncryptedText(findTestObject('Page_Login/input_Pwd'), '…')
WebUI.click(findTestObject('Page_Login/span_Login'))
WebUI.switchToFrame(findTestObject('Page_Home/iframe_main'), 10)
WebUI.click(findTestObject('Page_Home/a_Bond Trading'))
WebUI.click(findTestObject('Page_Home/a_Customer Service'))
WebUI.click(findTestObject('Page_Home/a_Adhoc Statement'))
WebUI.setText(findTestObject('Page_Adhoc/input_CIF'), '000002')
WebUI.click(findTestObject('Page_Adhoc/span_Search'))
```

KEYWORD-DRIVEN · this project

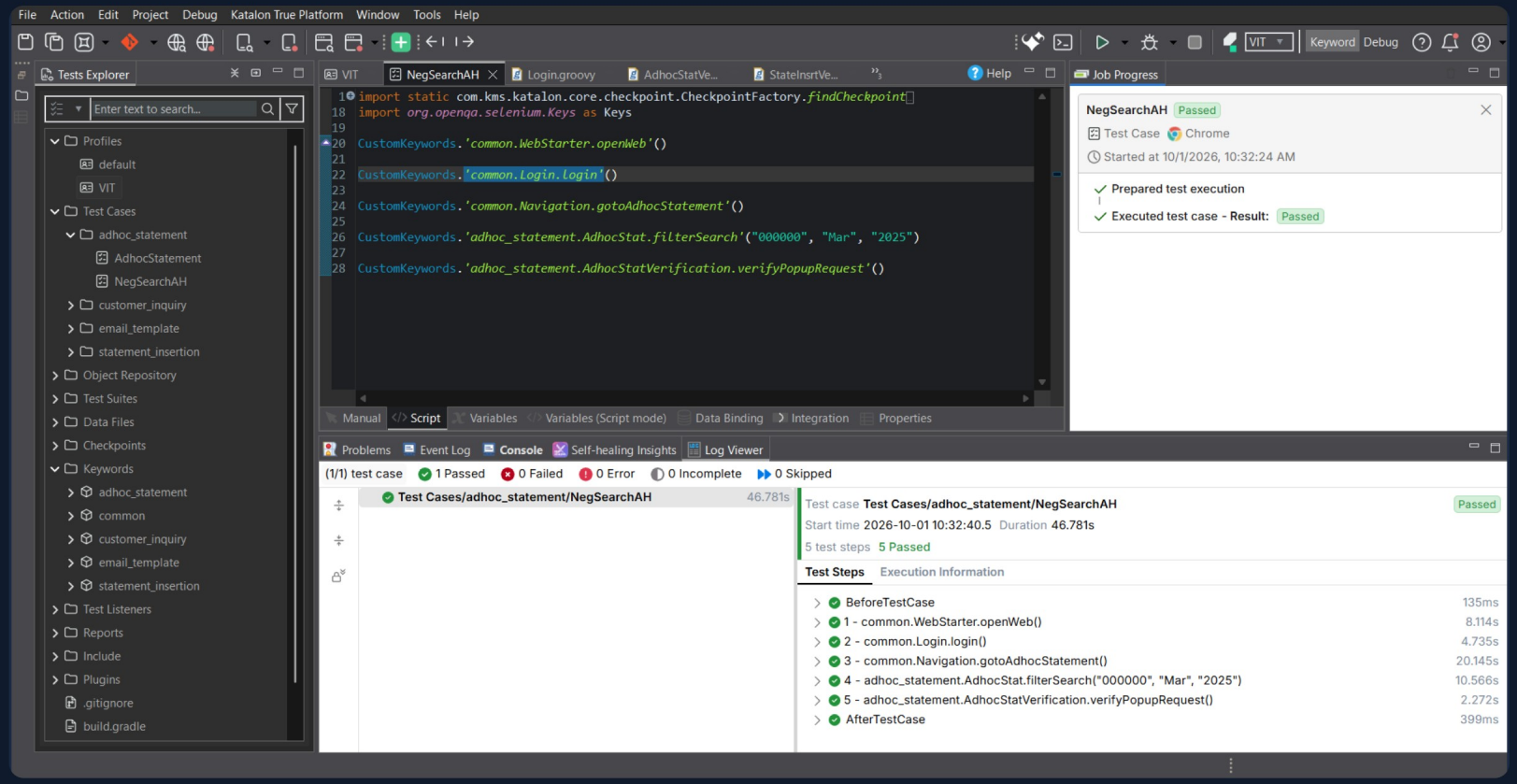
```
WebStarter.openWeb()
Login.login()
Navigation
.goToAdhocStatement()
AdhocStat.filterSearch(
cif, month, year)
```

URL and users come from the profile. Frames, menus and waits live inside the keywords.

Menu changes? Recorded: edit every test. Keywords: edit one Navigation method.

New environment? Recorded: URL and users are hardcoded. Keywords: switch profile.

Running in Katalon Studio



5 / 5

test steps passed

Chrome

browser run, VIT profile

Negative search: the test checks that the system raises a popup, and every log step is a named keyword.

Automation the whole QA team could build on

Reusable

Fix a keyword once and every script that calls it is updated.

Maintainable

Locators and environment data live outside the test logic.

Adopted

Introduced structured automation practices and worked with the QA team to standardize them.

SKILLS SHOWN IN THIS PROJECT

- Requirement analysis (FSD)
- Test case design
- Negative and boundary testing
- Role-based access testing
- Maker-checker workflows
- Katalon Studio
- Groovy
- SQL data validation
- Banking domain

Let's connect

Mulla Shadra · Software Quality Assurance

linkedin.com/in/mulla-shadra
mshadra6@gmail.com
github.com/MullaSh/katalon-keyword-driven-framework

Client, product and employer names are anonymized. No confidential data is shown.